wxcc-skills User Guide

Administer a Webex Contact Center tenant by talking to Claude.

This repo gives Claude Code a set of **MCP tools** for the WxCC Administration APIs, plus a library of **skills** — runbooks that teach Claude when and how to use them. You type plain English; Claude picks the skill, calls the tools, and shows you the results.

> Every API call these tools make was executed against a live tenant before it was written down. Anything unverified is labeled a **candidate** and says so.

What you can ask

You say	What happens
"How many users do we have? Who's on team X?"	Reads via the Users/Teams tools
"Which entry point does +1 719 555 0100 route to?"	Dial-number → entry-point lookup
"Create a team called Billing on the Denver site"	Shows a dry run, waits for your yes, creates it, re-reads to prove it
"Move agent Jane to the Sales team"	Updates the user's `teamIds` (site/type rules enforced)
"Add a wrap-up code called Escalated"	Creates the auxiliary code
"Raise queue X's service level threshold to 30 seconds"	Dry run → confirm → write → re-read diff
"How many calls did we take this week, by queue?"	GraphQL aggregation — grouped counts without transferring the tasks
"Delete team X"	Refuses if anything still references it, and lists what

Setup from scratch

Prerequisites: Python 3.10+, Claude Code, a WxCC **administrator** account, and rights to create an Integration at developer.webex.com.

1. Clone and install

```
git clone https://github.com/dwolgast-lab/wxcc-skills
cd wxcc-skills
pip install -r requirements.txt
```

The MCP server needs the `mcp` package. (`wxcc.py` itself is stdlib-only, so the CLI works without this — but the tools do not.)

2. Register an Integration

At developer.webex.com → **Manage Apps** → **Create an Integration**:

- **Redirect URI:** `http://localhost:8484/callback`
- **Scopes:** `cjp:config_read cjp:config cjp:config_write` (read-only? just the first)

Keep the client ID and secret.

3. Configure

```
cp .env.example .env
```

Fill in `WXCC_CLIENT_ID`, `WXCC_CLIENT_SECRET`, and `WXCC_API_BASE` — **the API host is region-specific** (`api.wxcc-us1.cisco.com`, `-eu1`, `-anz1`, …). Only `us1` has been exercised by this project.

Optionally set `WXCC_TENANT_LABEL` and `WXCC_TENANT_ALIASES` so Claude knows what you call this tenant.

4. Authenticate

```
python wxcc.py auth login
```

> ⚠️ **Open the printed URL in a private/incognito window yourself.** If your browser already holds a Webex session it will silently reuse that identity and mint a token for **the wrong tenant** — and it will look like it worked. This has bitten us three times. Setting `prompt=login` is not enough.

Verify — and do not skip this:

```
python wxcc.py auth status     # expect: valid, the granted scopes, and the right org_id
```

5. Register the MCP server

```
cp .mcp.json.example .mcp.json
```

Edit it to name your tenants **the way you talk about them** (`wxcc-acme`, not `wxcc-org2`):

```
{ "mcpServers": {
    "wxcc-sandbox": { "type": "stdio", "command": "python", "args": ["mcp_server.py"] }
} }
```

`.mcp.json` is gitignored — server names tend to be real customer names.

6. Approve and restart

```
claude          # interactive: review and approve the project MCP server, then /exit
```

Project MCP servers require an interactive approval — Claude cannot approve itself, and a non-interactive shell cannot either. **Renaming a server invalidates its approval**, so this step repeats if you rename one. This gate is specific to project scope (`.mcp.json` is checked-in, so it is treated as untrusted input); a `--scope user` server has no approval step at all — see the teammate path below.

Then restart Claude Code so it loads the server.

7. Confirm

Ask Claude: **"run wxcc_whoami"**. You should get the tenant's own name, its org id, whether it is production or a trial, and a live `HTTP 200` read check.

If the org id is not the tenant you meant, stop and redo step 4 in a private window.

The safety model

Reads are free. Writes are **mechanically gated** — this is enforced by the server, not a promise Claude makes:

- **Dry run by default.** A write call without `confirm` writes *nothing*. It returns the tenant, a field-level diff (or the object that would be destroyed), and the rollback.
- **The tenant is the first field of every write result**, tagged `[PRODUCTION]` or `[trial/sandbox]` — read from the tenant's own record, so it cannot drift from a stale label.
- **Every confirmed write re-reads and diffs.** This API can return **200 while silently ignoring a field** — a lesson learned live — so the tool reports `SILENTLY_IGNORED` when the API lied. "The API said OK" is never the proof.
- **Deletes pre-flight references** — via the API's own `incoming-references`, so the list of blockers is authoritative rather than guessed — and refuse with what to fix, rather than letting you discover the 412

afterward.

- **Required fields are checked before the call**, so you get a useful error instead of the API's 400.

Credentials live in a gitignored `.env`; tokens in a gitignored `.wxcc/`. Nothing secret is ever committed.

More than one tenant

There is deliberately no "switch tenant" command. A mutable *current tenant* is how a delete meant for sandbox lands on production. Instead each tenant is its own MCP server, so the tenant is **part of the tool name** and acting on the wrong one means calling a differently-named tool.

Each tenant gets a **profile** — its own config and its own token store, which never mix:

`WXCC_PROFILE`	config	tokens
(unset)	`.env`	`.wxcc/tokens.json`
`acme`	`.env.acme`	`.wxcc/tokens.acme.json`

```
# PowerShell has no inline env-var prefix - set it, then run
$env:WXCC_PROFILE = "acme"
python wxcc.py auth login       # private window, as THAT tenant's admin
python wxcc.py auth status      # org_id must be unique across profiles
Remove-Item Env:WXCC_PROFILE
```

```
{ "mcpServers": {
    "wxcc-sandbox": { "type": "stdio", "command": "python", "args": ["mcp_server.py"] },
    "wxcc-acme":    { "type": "stdio", "command": "python", "args": ["mcp_server.py"],
                      "env": { "WXCC_PROFILE": "acme" } } } }
```

A profile is only a label — the tenant is decided by whoever consents in the browser. Copying a token file to a new profile name gives you a live credential for the *wrong* tenant wearing the *right* name. Copy `.env.<profile>`; never copy tokens.

The profile name must match the `.env.<profile>` filename exactly.

Automatic guard: two profiles resolving to the same org id is almost always a wrong-tenant login. `auth login` refuses it, `auth status` warns, and `wxcc_whoami` returns a `WRONG_TENANT_WARNING`.

Two things that vary per tenant and are easy to miss:

- **Region** — `WXCC_API_BASE`. Only `us1` has been exercised here.
- **The Integration** — one client ID usually works across orgs, but a corporate Control Hub can **block third-party integrations**. If consent fails or returns no CC scopes, register an Integration inside that tenant and give that profile its own credentials.

Optional: run it in the cloud

`mcp_http.py` + `Dockerfile` deploy the same tools to Cloud Run, so teammates can use them **without cloning the repo, installing Python, or holding your credentials**.

The design point: **the server stores no Webex credentials at all.** Each caller runs their own Webex OAuth flow (Claude Code does this natively) and their token arrives on the request, gets used, and is forgotten. No token store, no refresh race, no standing admin credential on the internet. Each teammate consents as themselves, so your tenant's own RBAC and audit trail do the work.

```
gcloud run deploy wxcc-mcp --source . --region us-central1 \
  --allow-unauthenticated --max-instances=1 \
  --set-env-vars "^##^WXCC_ALLOWED_ORGS=<your org ids>##WXCC_API_BASE=<your region host>"
# then set WXCC_PUBLIC_URL to the service URL it prints, and redeploy
```

The service is public **because** Claude Code's OAuth token uses the `Authorization` header, which collides with Cloud Run IAM. An anonymous request gets 401 from the token verifier, and `WXCC_ALLOWED_ORGS` restricts it to your tenants. A caller's token only ever reaches its own org — the server grants no access they don't already have.

Client side, one entry per tenant, all pointing at the same service. **Declare the org each entry expects** with `?org=<org id>` — the server rejects a token from any other org with a `403 wrong_tenant`, which is what makes a wrong-tenant login *impossible* here rather than merely detectable:

```
{ "mcpServers": {
    "wxcc-cloud-acme": {
      "type": "http", "url": "https://<service>/mcp?org=<acme's org id>",
      "oauth": { "clientId": "<integration client id>", "callbackPort": 8484,
                 "scopes": "cjp:config_read cjp:config cjp:config_write" } } } }
```

Get the org id from `wxcc_whoami` (or `python wxcc.py auth status`). The header form `X-WXCC-Expected-Org` works too, but whether `headers` and `oauth` coexist in one entry is unverified — the URL form always works.

Tokens are stored per `<server name>|<config hash>`, so several entries can share one service URL and hold different tokens. **Changing an entry's URL changes the hash and orphans its token** — you will be asked to sign in again.

Store the secret and sign in:

```
MCP_CLIENT_SECRET=<secret> claude mcp add-json wxcc-cloud-acme '<the json above>' \
  --client-secret --scope project
claude mcp login wxcc-cloud-acme --no-browser    # paste the URL into a private window
```

Use `--no-browser`. It prints the authorize URL instead of opening one, which is the only reliable way to keep the browser from handing back the wrong identity.

The guard covers what the cloud otherwise cannot. A stateless server sees one token at a time, so it can never do the cross-profile comparison `wxcc.py` does locally. Declaring `?org=` replaces that with something

stronger: the check runs on **every request**, not just when someone remembers to look, and it cannot be satisfied by a token from another org because the org is read from the token itself.

Declare nothing and you keep the old behaviour — allowed, unguarded. On a real customer tenant, declare it.

Onboarding a teammate (they do NOT follow "Setup from scratch")

That section is for running it **locally**. A teammate using the cloud needs **no repo, no Python, no `.env`, no token store** — two commands:

```
MCP_CLIENT_SECRET=<integration secret> claude mcp add-json wxcc-acme `
  '{"type":"http","url":"https://<service>/mcp?org=<acme org id>","oauth":{"clientId":"<client id
>","callbackPort":8484,"scopes":"cjp:config_read cjp:config cjp:config_write"}}' `
  --client-secret --scope user

claude mcp login wxcc-acme --no-browser     # paste the URL into a private window
```

`--scope user` keeps it out of any repo directory, so it works from anywhere — and it needs **no interactive approval**: that gate exists only for project-scope `.mcp.json` servers. Confirmed live 2026-07-17 — a user-scope server's tool ran in a fresh non-interactive session with no approval recorded anywhere. The two commands above really are the whole setup.

Then: **"run wxcc_whoami"**. It must name the tenant they expect.

Three things that decide whether this works:

- **They must be a CC administrator of that tenant.** OAuth is per-user: they consent as themselves and get *their own* access. This does not lend them yours — that is the point. Their WxCC role, not this server, decides what they can do.
- **They need the Integration's client secret.** Webex offers no public-client option, so it lands in each teammate's keychain. Tolerable for a couple of people; at around five, put an OAuth shim in front (the server holds the secret and presents a public PKCE client).
- **Their org must be in `WXCC_ALLOWED_ORGS`** or every tool call 401s. Only you can change that — see below.

Adding a customer tenant to a cloud service

Steps 2 and 3 need `gcloud`, so **this is not self-service for a teammate**:

1. **Get the org id.** Chicken-and-egg: the allowlist needs it before anything works. Read it from Control Hub, from `wxcc_whoami` / `auth status` if anyone has local access — or let them try and read it out of the rejection: the server logs `reject: org <id> not in WXCC_ALLOWED_ORGS` to Cloud Run's stderr.
2. **Allowlist it** (must include the existing ids — this replaces the value):

`bash gcloud run services update wxcc-mcp --region us-central1 \ --update-env-vars "^##^WXCC_ALLOWED_ORGS=<existing ids>,<new id>"`

1. **Check the region.** `WXCC_API_BASE` is per-**service**, not per-caller, so a customer outside your service's region needs **its own Cloud Run service**. This is the sharpest limit in the current design.
2. They add their entry with `?org=<id>` and sign in.
3. Add a row to your alias table so Claude knows what people call it.

> The allowlist is **abuse prevention for your instance, not a data boundary** — a caller's token only ever reaches its own org, so a stranger with a valid token gains nothing they could not already do by calling Cisco directly. If teammate onboarding gets tiresome, dropping it and relying on the `?org=` guard is a defensible trade.

How it works

```
you  →  skill (when + judgement)  →  MCP tool (wxcc_list, wxcc_create, …)
                                            ↓
                                entity registry (paths, required fields, traps — enforced)
                                            ↓
                                wxcc.py (OAuth, tokens, requests — stdlib only)
                                            ↓
                                api.wxcc-<region>.cisco.com
```

Skills decide *when* and carry judgement the code can't. **The registry** holds the facts — which paths drop `v2`, which fields a create requires, which deletes get reference-blocked — so they're enforced rather than hoped for. **`wxcc.py`** owns OAuth and stays dependency-free, which is why the CLI still works with a bare Python install.

The CLI remains as an escape hatch for debugging. ⚠️ It uses whatever tenant `WXCC_PROFILE` resolves to in your shell — **not** the MCP server you were just talking to.

Tool catalog

Tool	Covers
`wxcc_whoami`	Which tenant am I actually on? Scopes, org, live read check
`wxcc_list` / `wxcc_get`	Read any of 22 entities
`wxcc_references`	What points **at** this object — the delete pre-flight, asked on its own, about something you intend to keep
`wxcc_create` / `wxcc_update` / `wxcc_delete`	Write one object, with dry-run + verify + reference-blocking
`wxcc_bulk_update` / `wxcc_bulk_create` / `wxcc_bulk_delete`	Write **many** objects of one entity in a single call, per-item 207 results — 19 entities, but **which of create/update/delete each supports differs sharply** and the tool refuses an unverified pair
`wxcc_list_entries` / `wxcc_add_entry` / `wxcc_update_entry` / `wxcc_remove_entry`	One entry at a time inside an address-book or outdial-ani
`wxcc_search_tasks`	Calls/tasks/agent sessions (GraphQL)
`wxcc_webhooks`	Event types + subscription CRUD

Entities: `user`, `team`, `site`, `contact-service-queue`, `entry-point`, `dial-number`, `skill`, `skill-profile`, `auxiliary-code`, `address-book`, `outdial-ani`, `agent-profile` (**Desktop Profile** — the old path name is backwards compatibility only), `desktop-layout`, `multimedia-profile`, `cad-variable` (**Global Variables**), `user-profile` (**admin access rights**, pinned to v3 — not the Desktop Profile), `resource-collection` (the scoped groupings a user profile points at), `business-hours`, `holiday-list`, `overrides` (the scheduling family — `business-hours.holidaysId` references a `holiday-list`), and `contact-number` (the caller-ID value shown on **internal** calls — despite the name, not the DID inventory; `number` is capped at 9 characters).

24 skills route to these — one per domain, split read/write where the risk differs, plus `wxcc-bulk` for many-at-once writes.

Known limits (honest list)

- **Users cannot be created or deleted** here (Control Hub owns identity); names/emails are read-only.
- **Phone numbers cannot be invented** — dial-number records map numbers already in the Webex Calling inventory. A fictional number returns 404, not 400.
- **Refused because unproven, not because impossible:** the `agent-profile` bulk/purge endpoints. The tools refuse rather than guess.
- **Webhook delivery payloads are unverified** — needs a real receiving endpoint.

- **Flows are out of scope by design.** Cisco ships its own `flow-store` MCP server; run it alongside this one. This repo owns the config flows bind to — entry points, queues, teams, skill profiles. A queue delete can orphan a flow, and these tools cannot see it.
- Only `us1` has been exercised.
- Anything marked **candidate** has not been run against a live tenant.

Roadmap

Bulk-export · webhook delivery payload (needs a receiver).

Draft 3 — 2026-07-16.